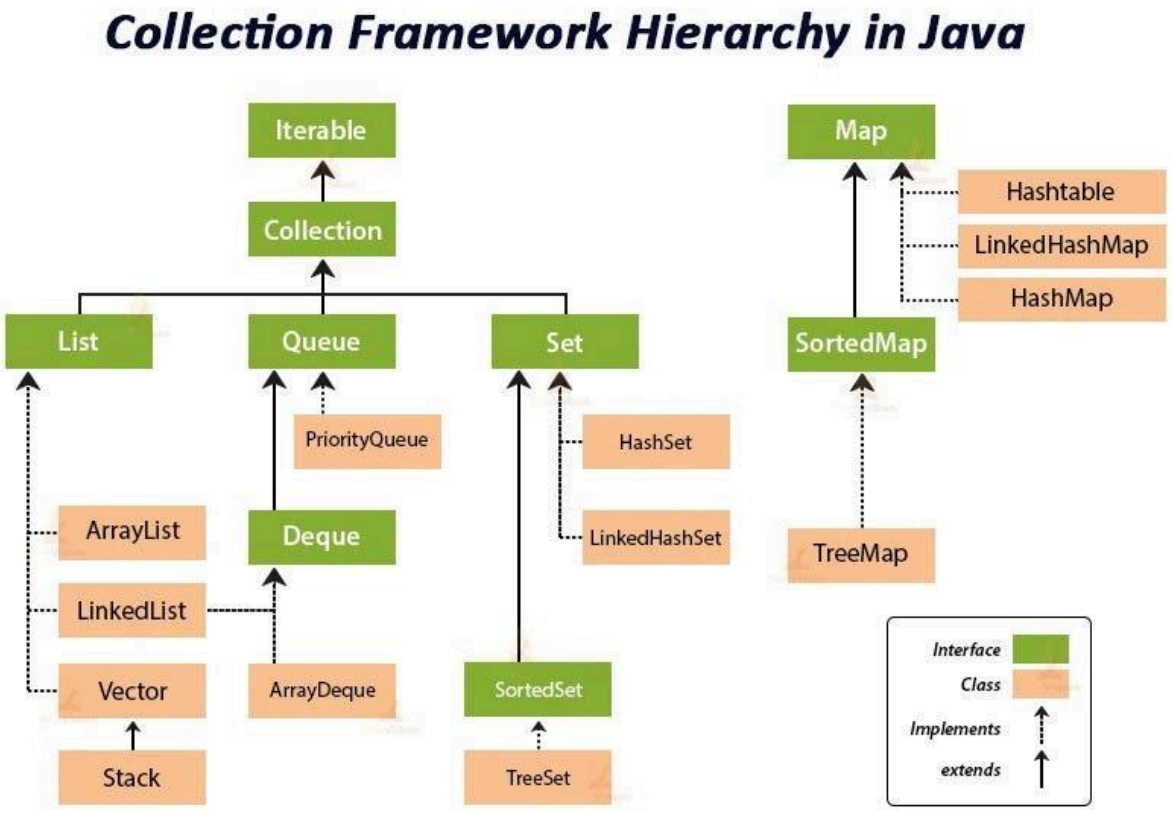
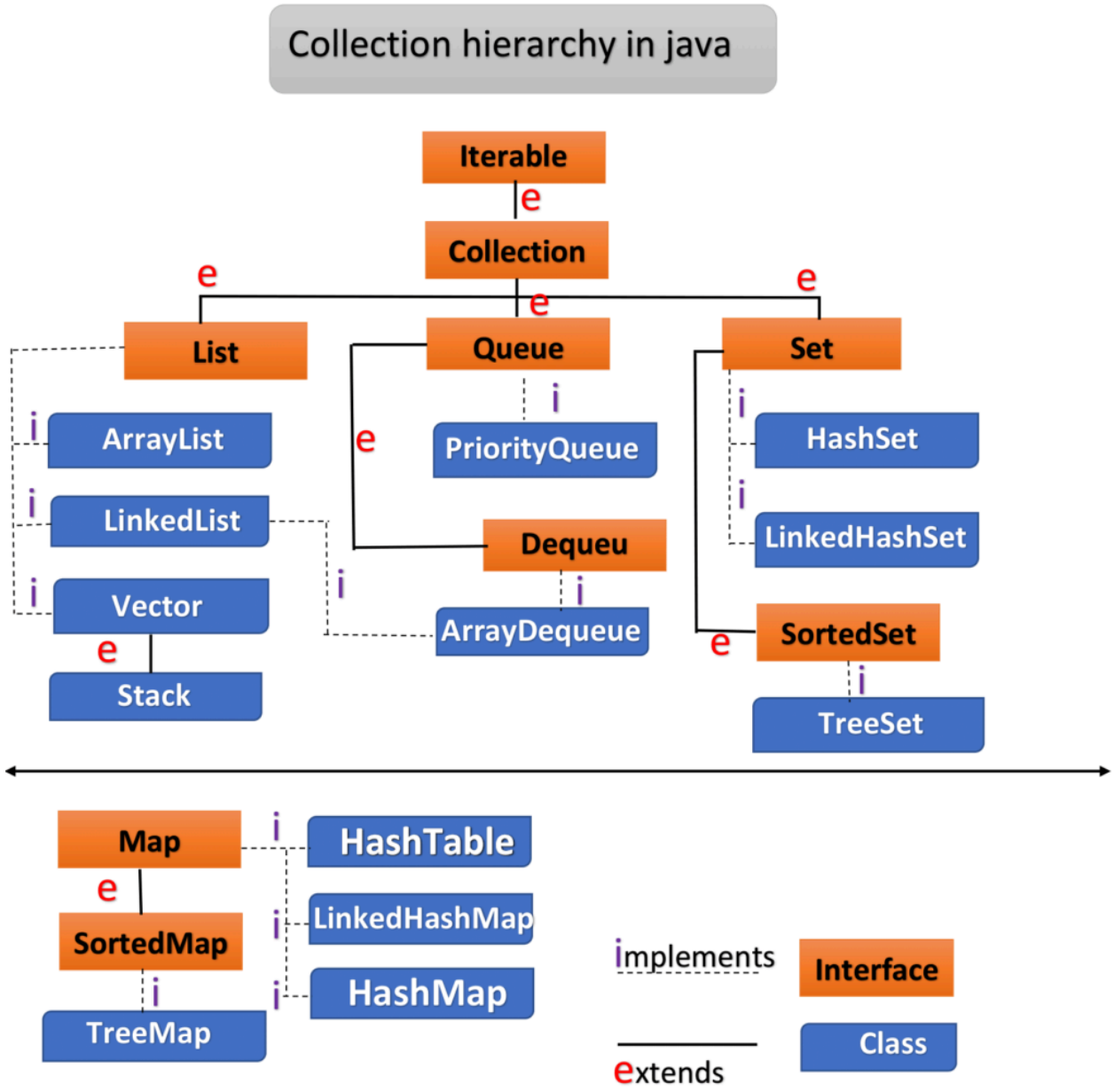
Web Dev Mastery

☕ Java Collection Framework (JCF)




Collection hierarchy in java
Iterable
e
Collection
e
e
e
List
Queue
Set
i
ArrayList
i
LinkedList
i
Vector
e
Stack
e
i
PriorityQueue
Dequeu
i
ArrayDequeue
i
HashSet
i
LinkedHashSet
e
SortedSet
i
TreeSet
Map
i
HashTable
e
SortedMap
i
LinkedHashMap
i
TreeMap
i
HashMap
implements
Interface
extends
Class

Operation \ List	LinkedList	ArrayList
get(int index)	O(n/4) average	O(1)
add(E element)	O(1)	O(1) amortized O(n) worst-case
add(int index, E element)	O(n/4) average O(1) if index is 0	O(n/2) average
remove	O(n/4) average	O(n/2) average
Iterator.remove()	O(1)	O(n/2) average
ListIterator.add(E element)	O(1)	O(n/2) average

4

✅ What is Java Collection Framework?

Java Collection Framework is a unified architecture in Java that provides **classes and interfaces** to store, manipulate, and process groups of objects efficiently.

👉 It is present in the package:

```
java.util
```

🎯 Why do we need Collection Framework?

Before collections, Java used **arrays**, which had limitations:

Arrays ❌	Collections ✅
Fixed size	Dynamic size
No built-in methods	Rich API methods

Less flexible	Highly flexible
Cannot grow/shrink	Can grow/shrink

🧱 Core Components of Collection Framework

1️⃣ Interfaces

They define **what** a collection can do.

- `Collection` (root interface)
- `List`
- `Set`
- `Queue`
- `Deque`
- `Map` (separate hierarchy)

2️⃣ Classes

They provide **how** the collection works.

- **List Implementations (Ordered, Allow Duplicates)**
  - `ArrayList`
  - `LinkedList`
  - `Vector`
  - `Stack`
- **Set Implementations (No Duplicates)**
  - `HashSet`
  - `LinkedHashSet`
  - `TreeSet`
- **Queue Implementations (FIFO)**
  - `PriorityQueue`
  - `ArrayDeque`

🔹 **Map Implementations (Key–Value)**

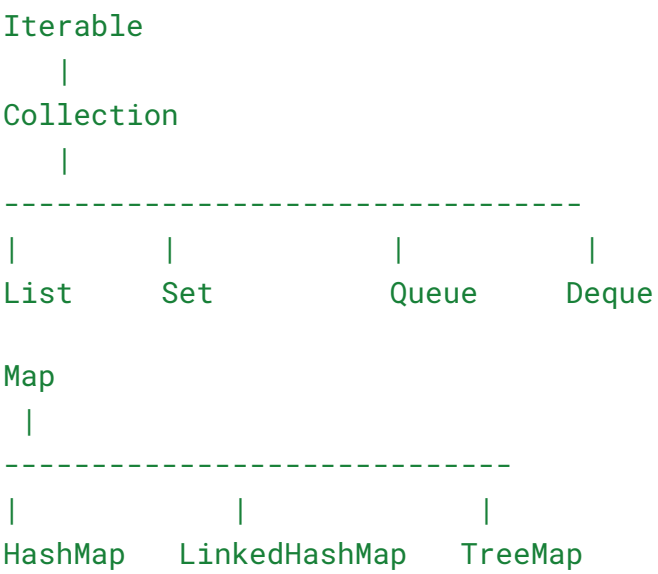
- `HashMap`
- `LinkedHashMap`
- `TreeMap`
- `Hashtable`

3️⃣ Algorithms (Utility Methods)

Provided via `Collections` class:

- `sort()`
- `reverse()`
- `shuffle()`
- `binarySearch()`
- `max()`, `min()`

🧩 Collection Hierarchy (Simplified)



🔹 Java `List` Interface – COMPLETE & IN-DEPTH GUIDE (Including Nested Lists)

1️⃣ What is `List` in Java?

`List` is an **interface** in the Java Collection Framework that represents an **ordered, index-based collection**.

```
List<E>
```

Core Properties

- ✔ Maintains **insertion order**
- ✔ **Allows duplicates**
- ✔ **Index-based access**
- ✔ Can store **null values**
- ✔ Part of `java.util` package

2️⃣ List Hierarchy (Internal View)

```
Iterable
   ↓
Collection
   ↓
List
   ↓
-----------------------------------------
|          |           |                |
ArrayList  LinkedList  Vector          Stack
```

3️⃣ Creating a List (Different Ways)

```
List<Integer> list = new ArrayList<>();
List<Integer> list2 = new LinkedList<>();
List<Integer> list3 = new Vector<>();
```

4️⃣ ALL `List` METHODS – DETAILED EXPLANATION

> `List` extends `Collection`, so it inherits **all Collection methods** + adds **index-based methods**

🔹 A. Adding Elements

`boolean add(E e)`

```
list.add(10);
```

`void add(int index, E element)`

```
list.add(1, 20);
```

`boolean addAll(Collection<? extends E> c)`

```
list.addAll(List.of(30, 40));
```

`boolean addAll(int index, Collection<? extends E> c)`

```
list.addAll(2, List.of(50, 60));
```

🔹 B. Accessing Elements

`E get(int index)`

```
int x = list.get(0);
```

🔹 C. Updating Elements

`E set(int index, E element)`

```
list.set(0, 100);
```

🔹 D. Removing Elements

`E remove(int index)`

```
list.remove(1);
```

`boolean remove(Object o)`

```
list.remove(Integer.valueOf(10));
```

`boolean removeAll(Collection<?> c)`

```
list.removeAll(List.of(20, 30));
```

`void clear()`

```
list.clear();
```

🔹 E. Searching Elements

`boolean contains(Object o)`

```
list.contains(10);
```

`int indexOf(Object o)`

```
list.indexOf(10);
```

`int lastIndexOf(Object o)`

```
list.lastIndexOf(10);
```

🔹 F. Size & Status

`int size()`

`boolean isEmpty()`

🔹 G. Iteration Techniques

```
// for-loop
for(int i = 0; i < list.size(); i++) {}

// enhanced for
for(int x : list) {}

// iterator
Iterator<Integer> it = list.iterator();

Iterator<Integer> it = list.iterator();

        while (it.hasNext()) {
            System.out.println(it.next());
        }
```

```
// listIterator (bi-directional)
ListIterator<Integer> li = list.listIterator();

// Java 8+
list.forEach(System.out::println);
```

🔹 H. Conversion Methods

```
Object[] arr = list.toArray();
Integer[] arr2 = list.toArray(new Integer[0]);
```

🔹 I. Sub List

`List<E> subList(int fromIndex, int toIndex)`

```
List<Integer> sub = list.subList(1, 4); // toIndex exclusive
```

⚠ SubList is a **view**, not a new list.

🔹 J. Retain Elements

```
list.retainAll(List.of(10, 20));
```

🔹 K. Replace & Sort (Java 8+)

```
list.replaceAll(x -> x * 2);
list.sort(Comparator.naturalOrder());
```

5️⃣🚀 NESTED LISTS (IMPORTANT & CONFUSING TOPIC)

🔹 What is `List<List<Integer>>`?

A **List inside another List**
Used to represent:

- 2D arrays / matrices
- Graph adjacency list
- Grouped data

```
List<List<Integer>> matrix = new ArrayList<>();
```

6️⃣ Creating a Nested List (Step-by-Step)

```
List<List<Integer>> list = new ArrayList<>();

list.add(new ArrayList<>(List.of(1, 2, 3)));
list.add(new ArrayList<>(List.of(4, 5, 6)));
list.add(new ArrayList<>(List.of(7, 8, 9)));
```

📌 Structure:

```
[
 [1, 2, 3],
 [4, 5, 6],
 [7, 8, 9]
]
```

7️⃣ Accessing Nested List Elements

```
int x = list.get(1).get(2); // 6
```

Explanation:

- `list.get(1)` → `[4,5,6]`
- `.get(2)` → `6`

8️⃣ Iterating Over Nested List

🔹 Using Loops

```
for(int i = 0; i < list.size(); i++) {
    for(int j = 0; j < list.get(i).size(); j++) {
        System.out.print(list.get(i).get(j) + " ");
    }
}
```

🔹 Enhanced For

```
for(List<Integer> row : list) {
    for(int val : row) {
        System.out.print(val + " ");
    }
}
```

9️⃣ Modifying Nested List

Add element

```
list.get(0).add(100);
```

Update element

```
list.get(1).set(2, 999);
```

Remove element

```
list.get(2).remove(1);
```

🔟 Dynamic Matrix (Rows & Columns Unknown)

```
List<List<Integer>> matrix = new ArrayList<>();

for(int i = 0; i < 3; i++) {
    matrix.add(new ArrayList<>());
    for(int j = 0; j < 3; j++) {
        matrix.get(i).add(i + j);
    }
}
```

1️⃣1️⃣ Nested List vs 2D Array

Feature	List<List<Integer>>	int[][]
Size	Dynamic	Fixed
Flexibility	High	Low
Methods	Many	Very few
Performance	Slightly slower	Faster

1️⃣2️⃣ Common Interview Mistakes 🚨

❌ Forgetting to initialize inner list
❌ IndexOutOfBoundsException
❌ Assuming fixed size
❌ Confusing deep copy vs shallow copy

1️⃣3️⃣ Real-World Use Cases

- Matrix problems (DSA)
- Graph adjacency list
- Grouping data (users by city)
- Dynamic tables

1️⃣4️⃣ Time Complexity (Nested List)

Operation	Time
Access	O(1)
Insert	O(n)
Traverse	O(rows × cols)

🔹 Stack in Java Collection Framework

In Java, **Stack** is a **class** present in the **Collection Framework** that follows the **LIFO principle** 👉 **Last In, First Out**

That means:

- The element inserted **last** is removed **first**
- Similar to a stack of plates 🍽️

🔹 Position of Stack in Collection Framework

```
Object
  └── Collection
       └── List
            └── Vector
                 └── Stack
```

✔ Stack **extends Vector**
✔ Stack **implements List**

So Stack supports:

- Index-based access
- Iteration
- All List & Vector methods + stack-specific methods

🔹 Import Statement

```
import java.util.Stack;
```

🔹 Creating a Stack

```
Stack<Integer> stack = new Stack<>();
```

🔹 Core Stack Methods (MOST IMPORTANT)

1️⃣ push() → Insert element at top

```
stack.push(10);
stack.push(20);
stack.push(30);
```

Stack now:

```
Top → 30
       20
       10
```

2️⃣ pop() → Remove & return top element

```
int removed = stack.pop(); // 30
```

✔ Removes **top element**
❌ Throws `EmptyStackException` if stack is empty

3️⃣ peek() → View top element (no removal)

```
int top = stack.peek(); // 20
```

✔ Safe way to check top element

4️⃣ empty() → Check if stack is empty

```
boolean isEmpty = stack.empty(); // false
```

5️⃣ search() → Position from top (1-based index)

```
int pos = stack.search(10); // 2
```

- Top element position = 1
- Returns `-1` if element not found

🔹 Complete Example

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        stack.push(10);
        stack.push(20);
        stack.push(30);

        System.out.println(stack);        // [10, 20, 30]
        System.out.println(stack.peek()); // 30
        System.out.println(stack.pop());  // 30
        System.out.println(stack);        // [10, 20]
        System.out.println(stack.search(10)); // 2
        System.out.println(stack.empty());    // false
    }
}
```

🔹 Iterating Stack (3 Ways)

🔸 Using for-each loop

```
for (int x : stack) {
    System.out.println(x);
}
```

🔸 Using Iterator

```
Iterator<Integer> it = stack.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

🔸 Using index

```
for (int i = 0; i < stack.size(); i++) {
    System.out.println(stack.get(i));
}
```

🔹 Stack Allows Duplicate & Null?

✔ **Duplicates** → Allowed
✔ **Null** → Allowed

```
stack.push(null);
stack.push(10);
stack.push(10);
```

🔹 Important Notes (INTERVIEW ⭐)

❗ Stack is Legacy Class

- Stack is **synchronized**
- Slower than modern alternatives

✅ Recommended Alternative

Use **Deque** instead of Stack 👇

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.pop();
stack.peek();
```

✔ Faster
✔ Modern
✔ Preferred in real projects

🔹 Stack vs Deque (Quick Comparison)

Feature	Stack	Deque
Thread-safe	Yes	No
Performance	Slow	Fast
Legacy	Yes	No
Recommended	❌	✅

🔹 Real-Life Use Cases

✔ Undo / Redo operations
✔ Expression evaluation
✔ Function call stack
✔ Browser back/forward
✔ DFS (Depth First Search)

🔹 One-Line Summary (Exam Ready ✅)

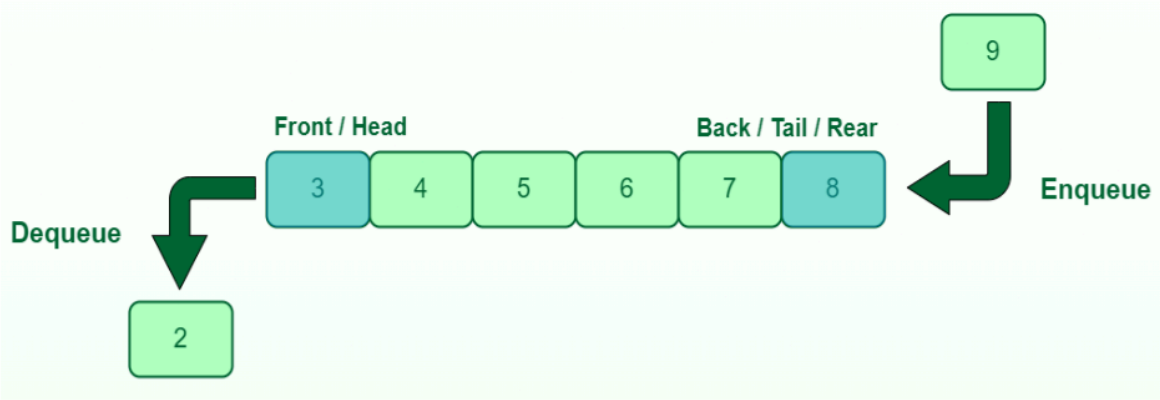
Stack is a LIFO data structure in Java, implemented as a class that extends Vector and provides methods like push(), pop(), peek(), empty(), and search().

🔹 Queue in Java Collection Framework

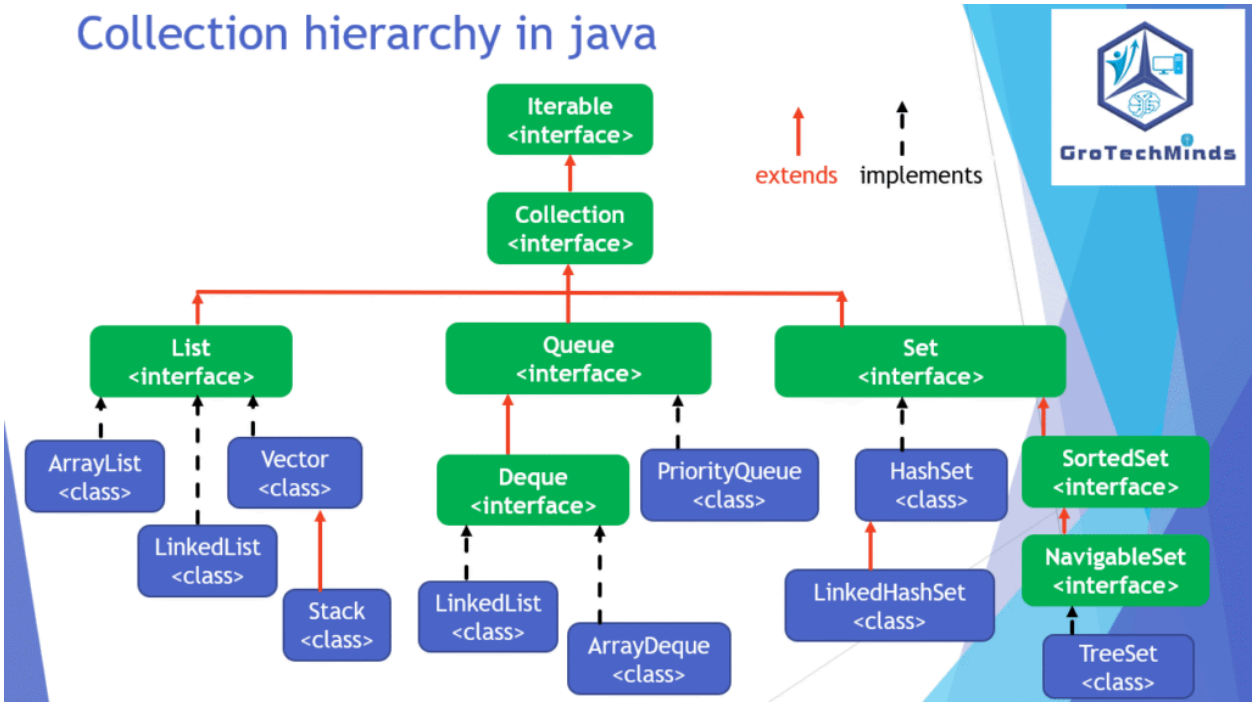
Queue is a **data structure & interface** in Java that follows the **FIFO principle**
👉 **First In, First Out**

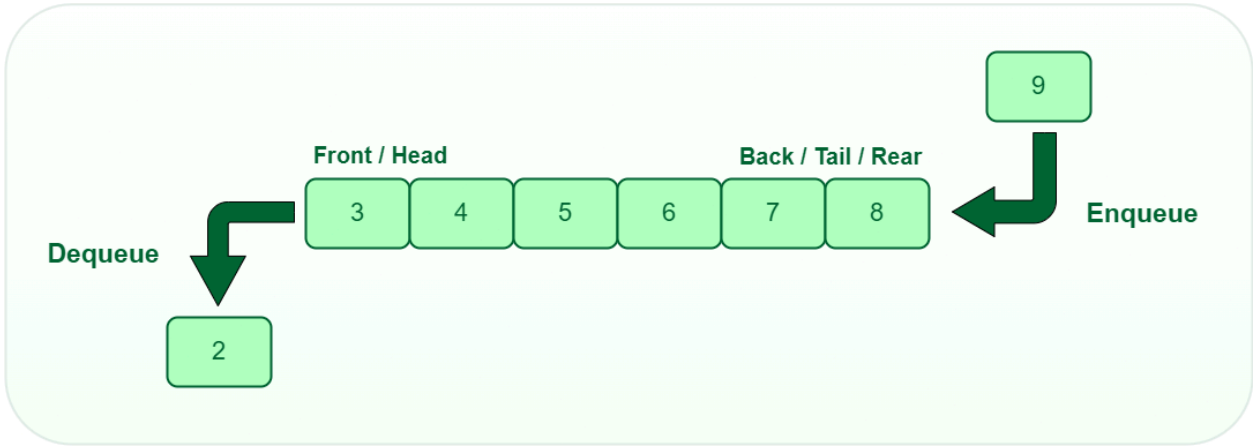
That means:

- The element inserted **first** is removed **first**
- Just like a **line at a ticket counter** 🎟️



Queue Data Structure





🔹 Position of Queue in Collection Framework

```
Object
 └── Collection
      └── Queue   (Interface)
           ├── PriorityQueue
           └── Deque (Interface)
                ├── ArrayDeque
                └── LinkedList
```

✔ **Queue is an interface**, not a class
✔ Implemented by:

- `LinkedList`
- `PriorityQueue`
- `ArrayDeque` (via Deque)

🔹 Import Statement

```
import java.util.Queue;
import java.util.LinkedList;
```

🔹 Creating a Queue

```
Queue<Integer> q = new LinkedList<>();
```

✔ This is the **most common** way

🔹 Core Queue Operations (VERY IMPORTANT ⭐)

1️⃣ add() → Insert element

```
q.add(10);
q.add(20);
q.add(30);
```

Queue now:

```
Front → 10 | 20 | 30 ← Rear
```

❌ Throws exception if insertion fails

2️⃣ offer() → Insert element (safe)

```
q.offer(40);
```

✔ Returns `true / false`
✔ Preferred in real projects

3️⃣ remove() → Remove front element

```
int x = q.remove(); // 10
```

❌ Throws exception if queue is empty

4️⃣ poll() → Remove front element (safe)

```
int x = q.poll(); // 10
```

✔ Returns `null` if queue is empty
✔ **Most used method**

5️⃣ element() → View front element

```
int front = q.element();
```

❌ Exception if empty

6️⃣ peek() → View front element (safe)

```
int front = q.peek();
```

✔ Returns `null` if empty
✔ **Preferred**

🔹 Complete Example

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();

        q.offer(10);
        q.offer(20);
        q.offer(30);

        System.out.println(q);         // [10, 20, 30]
        System.out.println(q.peek()); // 10
        System.out.println(q.poll()); // 10
        System.out.println(q);         // [20, 30]
    }
}
```

🔹 Iterating Queue

🔸 For-each loop

```
for (int x : q) {
    System.out.println(x);
}
```

🔸 Using Iterator

```
Iterator<Integer> it = q.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

🔹 Does Queue Allow?

Feature	Allowed
Duplicate elements	✅ Yes
Null elements	⚠ Depends

- `LinkedList` → allows **null**
- `PriorityQueue` → ❌ no null
- `ArrayDeque` → ❌ no null

🔹 Important Queue Implementations

🔹 1. LinkedList (Normal FIFO Queue)

```
Queue<Integer> q = new LinkedList<>();
```

✔ Maintains insertion order
✔ Allows duplicates

🔹 2. PriorityQueue (Priority-based)

```
Queue<Integer> pq = new PriorityQueue<>(); // min heap

pq.offer(30);
pq.offer(10);
pq.offer(20);
System.out.println(pq); // [10, 30, 20]
```

✔ Smallest element comes first
❌ No fixed FIFO order

```
  Queue<Integer>q = new
PriorityQueue<>(Comparator.reverseOrder());
```

// max heap

🔹 3. ArrayDeque (Best Choice ⭐)

```
Deque<Integer> dq = new ArrayDeque<>();
dq.offer(10);
dq.offer(20);
dq.poll();
```

✔ Faster than LinkedList
✔ No capacity issue
✔ Used as **Queue + Stack**

🔹 Queue vs Stack (INTERVIEW 🔥)

Feature	Queue	Stack
Order	FIFO	LIFO
Insert	Rear	Top
Remove	Front	Top
Interface/Class	Interface	Class
Real use	Scheduling	Undo/Redo

🔹 Real-Life Use Cases

✔ CPU Scheduling
✔ Printer queue
✔ Order processing
✔ BFS (Breadth First Search)
✔ Request handling in servers

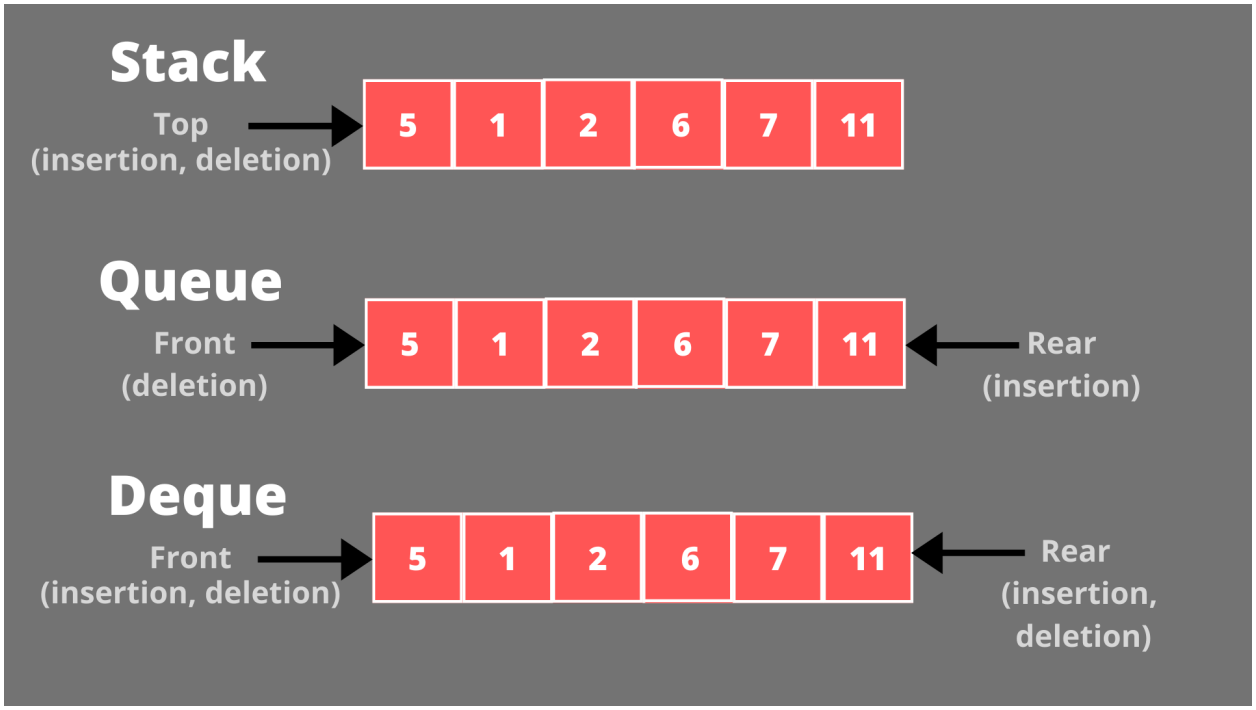
🔹 One-Line Summary (Exam Ready ✅)

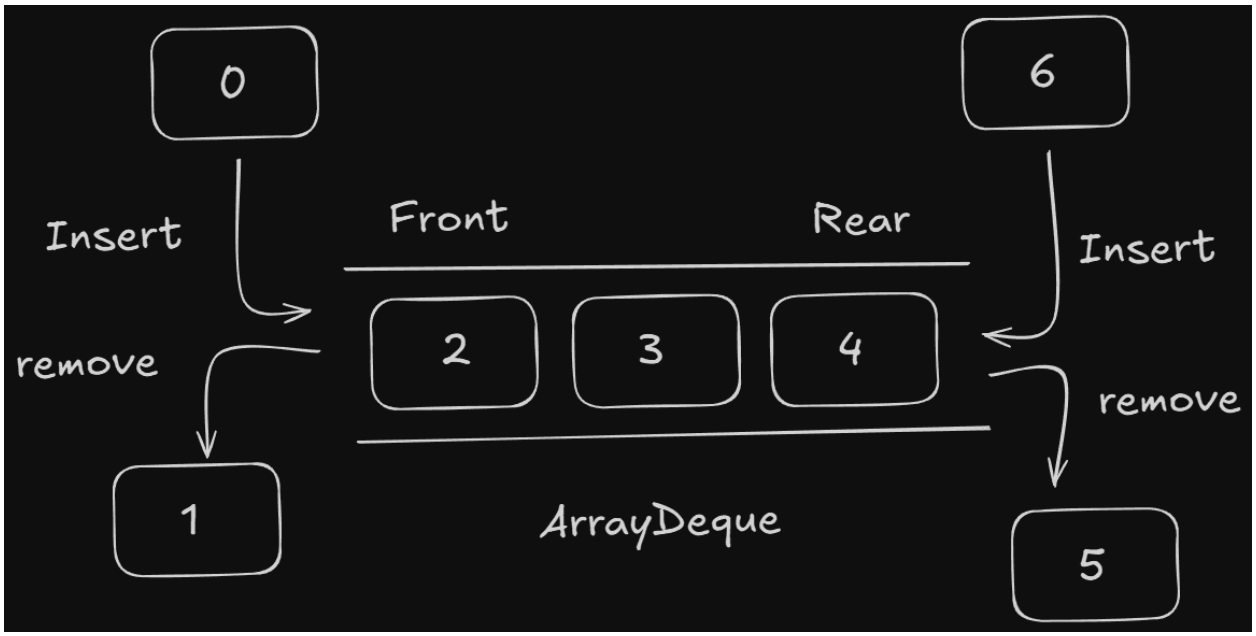
Queue is a FIFO data structure in Java represented by an interface, commonly implemented using LinkedList, PriorityQueue, or ArrayDeque, supporting operations like offer(), poll(), and peek().

🔹 ArrayDeque in Java Collection Framework

ArrayDeque is a **resizable array-based** implementation of the **Deque interface** in Java. It can work as **both Stack (LIFO)** and **Queue (FIFO)** and is the **most recommended** alternative to `Stack` and `LinkedList`.







🔹 Position in Collection Framework

```
Object
 └── Collection
       └── Queue
             └── Deque (Interface)
```

```
        └── ArrayDeque (Class)
```

✔ Implements **Deque**
✔ Not a legacy class
✔ High performance

🔹 Import Statement

```
import java.util.ArrayDeque;
import java.util.Deque;
```

🔹 Creating ArrayDeque

```
Deque<Integer> dq = new ArrayDeque<>();
```

✅ **Best practice**: program to interface (`Deque`)

🔹 Internal Working (IMPORTANT ⭐)

ArrayDeque uses a **circular dynamic array**:

```
[ _ | 10 | 20 | 30 | _ | _ ]
      ↑            ↑
    head         tail
```

✔ No shifting of elements
✔ Uses modulo arithmetic
✔ Automatically resizes (grows)

🔹 Core Deque Operations

🔸 Add Elements

```
dq.addFirst(10);   // front
dq.addLast(20);    // rear
dq.offerFirst(5);
dq.offerLast(30);
```

🔸 Remove Elements

```
dq.removeFirst();   // removes front
dq.removeLast();    // removes rear

dq.pollFirst();     // safe (null if empty)
dq.pollLast();
```

🔸 Peek Elements

```
dq.getFirst();
dq.getLast();

dq.peekFirst();     // safe
dq.peekLast();
```

🔹 ArrayDeque as Queue (FIFO)

```
Deque<Integer> q = new ArrayDeque<>();

q.offer(10);
q.offer(20);
q.offer(30);

q.poll();   // 10
q.peek();   // 20
```

✔ Faster than `LinkedList`

🔹 ArrayDeque as Stack (LIFO)

```
Deque<Integer> stack = new ArrayDeque<>();

stack.push(10);
stack.push(20);
stack.push(30);

stack.pop();   // 30
stack.peek();  // 20
```

✔ Recommended replacement for `Stack`

🔹 Iterating ArrayDeque

🔸 for-each

```
for (int x : dq) {
    System.out.println(x);
}
```

🔸 Iterator

```
Iterator<Integer> it = dq.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

🔸 Descending Iterator

```
Iterator<Integer> it = dq.descendingIterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

🔹 Does ArrayDeque Allow?

Feature	Allowed
Duplicate values	✅ Yes
Null values	❌ No
Thread-safe	❌ No

🔹 Time Complexity (INTERVIEW ⭐)

Operation	Time
Add/remove first	O(1)
Add/remove last	O(1)
Search	O(n)

🔹 ArrayDeque vs Stack vs LinkedList (🔥)

Feature	Stack	LinkedList	ArrayDeque
Legacy	✅	❌	❌
Performance	❌ Slow	⚠ Medium	✅ Fast
Null allowed	✅	✅	❌
Thread-safe	✅	❌	❌
Recommended	❌	⚠	✅

🔹 When to Use ArrayDeque?

✔ Use as **Stack** instead of `Stack`
✔ Use as **Queue** instead of `LinkedList`

✔ High-performance applications
✔ DSA problems (BFS, DFS, sliding window)

🔹 Common Interview Questions ⭐

❓ Why ArrayDeque is faster than LinkedList?
👉 No node objects, better cache locality.

❓ Why null is not allowed?
👉 To avoid ambiguity in `poll()` and `peek()`.

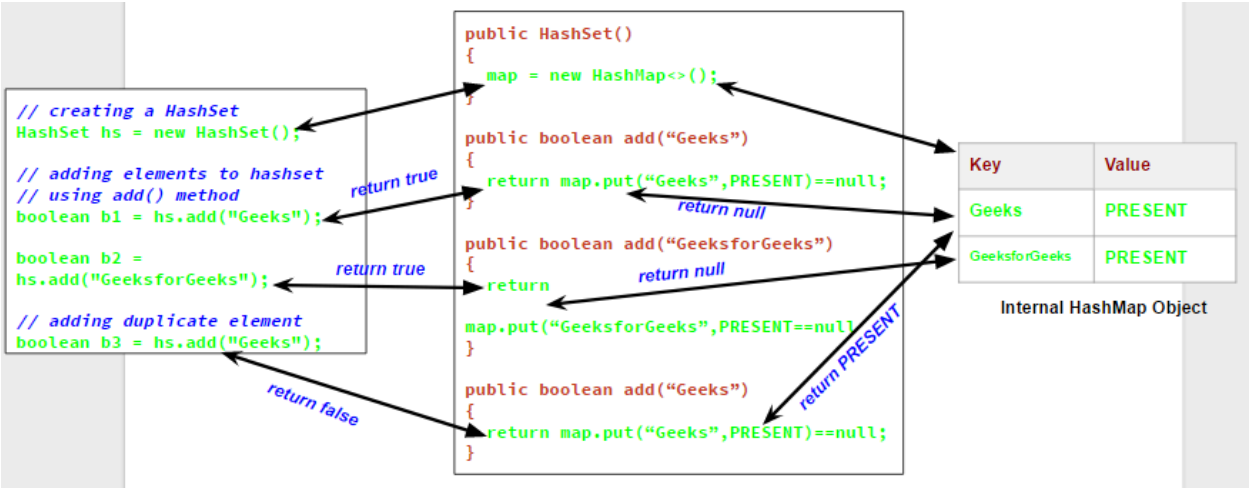
🔹 One-Line Summary (Exam Ready ✅)

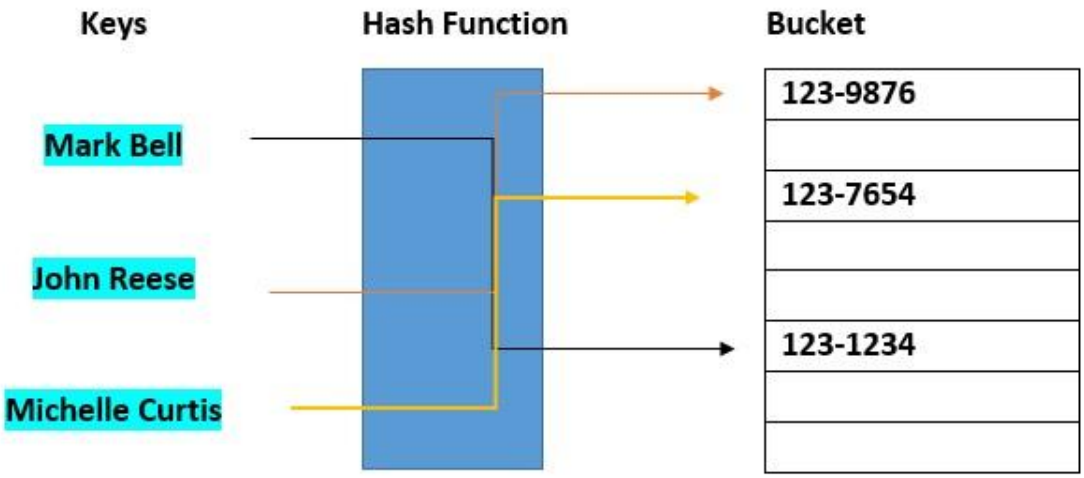
> **ArrayDeque is a high-performance, resizable circular array implementation of Deque in Java, recommended for both stack and queue operations.**

🔹 HashSet in Java Collection Framework

HashSet is a **class** in Java that implements the **Set interface** and stores **unique elements only**.
It internally uses **hashing** for fast operations.

👉 **No duplicates**
👉 **No guaranteed order**
👉 **Allows only one null value**







🔹 Position in Collection Framework

```
Object
 └── Collection
       └── Set (Interface)
             └── HashSet (Class)
```

✔ HashSet implements **Set**
✔ Backed by **HashMap internally**

🔹 Import Statement

```
import java.util.HashSet;
import java.util.Set;
```

🔹 Creating HashSet

```
Set<Integer> set = new HashSet<>();
```

✅ **Best practice**: program to interface (`Set`)

🔹 Key Characteristics (INTERVIEW ⭐)

Feature	HashSet
Duplicates	❌ Not allowed
Order	❌ No order
Null values	✅ Only one
Thread-safe	❌ No
Performance	✅ Fast (O(1))

🔹 Internal Working (IMPORTANT 🔥)

HashSet internally uses a **HashMap**:

```
HashSet<Integer> set = new HashSet<>();
```

is internally similar to:

```
HashMap<Integer, Object> map = new HashMap<>();
```

- **Element → key**
- **Dummy object → value**

How insertion works:

1. `hashCode()` is called
2. Bucket index is calculated
3. `equals()` is used to avoid duplicates

🔹 Adding Elements

```
set.add(10);
set.add(20);
set.add(30);
set.add(10);  // duplicate (ignored)
```

Output:

```
[20, 10, 30]   // order not fixed
```

🔹 Removing Elements

```
set.remove(20);
```

🔹 Checking Element

```
set.contains(10); // true
```

🔹 Size & Clear

```
set.size();
set.clear();
```

🔹 Iterating HashSet (IMPORTANT)

🔸 for-each loop

```
for (int x : set) {
    System.out.println(x);
}
```

🔸 Iterator

```
Iterator<Integer> it = set.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

❌ No index-based access (because no order)

🔹 Time Complexity (INTERVIEW ⭐)

Operation	Time
add()	O(1)
remove()	O(1)
contains()	O(1)

(Worst case: O(n) due to collisions)

🔹 HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Order	❌ No	✅ Insertion	✅ Sorted
Speed	✅ Fastest	⚠ Medium	❌ Slow
Null	✅ One	✅ One	❌ No

Set<Integer> set = new LinkedHashSet<>();

```
set.add(10);
set.add(20);
set.add(30);

System.out.println(set); // [10, 20, 30]
```

🔹 LinkedHashSet Properties

Feature	LinkedHashSet
Duplicates	❌ Not allowed
Order	✅ Insertion order
Null values	✅ One

Performance	⚠ Slightly slower
Thread-safe	❌ No

🔹 Time Complexity

Operation	Time
add	O(1)
remove	O(1)
contains	O(1)

🔹 When to Use LinkedHashSet?

✔ Need **unique + ordered data**
✔ History / logs
✔ Predictable iteration order

🔹 3. TreeSet (Sorted Set)

✅ What is TreeSet?

TreeSet stores elements in **sorted order** (ascending by default).

✔ Unique elements
✔ Sorted automatically
✔ Uses **Red-Black Tree**

🔹 Internal Working of TreeSet (IMPORTANT 🔥)

- Based on **Self-balancing Red-Black Tree**

- Sorting uses:
    - `Comparable` OR
    - `Comparator`

🔹 Example (Natural Sorting)

```
Set<Integer> set = new TreeSet<>();

set.add(30);
set.add(10);
set.add(20);

System.out.println(set); // [10, 20, 30]
```

🔹 TreeSet with Comparator

```
Set<Integer> set = new TreeSet<>((a, b) -> b - a);

set.add(10);
set.add(20);
set.add(30);

System.out.println(set); // [30, 20, 10]
```

🔹 TreeSet Properties

Feature	TreeSet
Duplicates	❌ Not allowed
Order	✅ Sorted

Null values	❌ Not allowed
Performance	❌ Slower
Thread-safe	❌ No

🔹 Time Complexity

Operation	Time
add	O(log n)
remove	O(log n)
contains	O(log n)

🔹 When to Use TreeSet?

✔ Need **sorted data**
✔ Range queries
✔ Leaderboards / rankings

🔥 HashSet vs LinkedHashSet vs TreeSet (VERY IMPORTANT)

Feature	HashSet	LinkedHashSet	TreeSet
Order	❌ No	✅ Insertion	✅ Sorted
Speed	✅ Fastest	⚠ Medium	❌ Slow
Null allowed	✅ One	✅ One	❌ No
Internal DS	HashTable	HashTable + DLL	Red-Black Tree

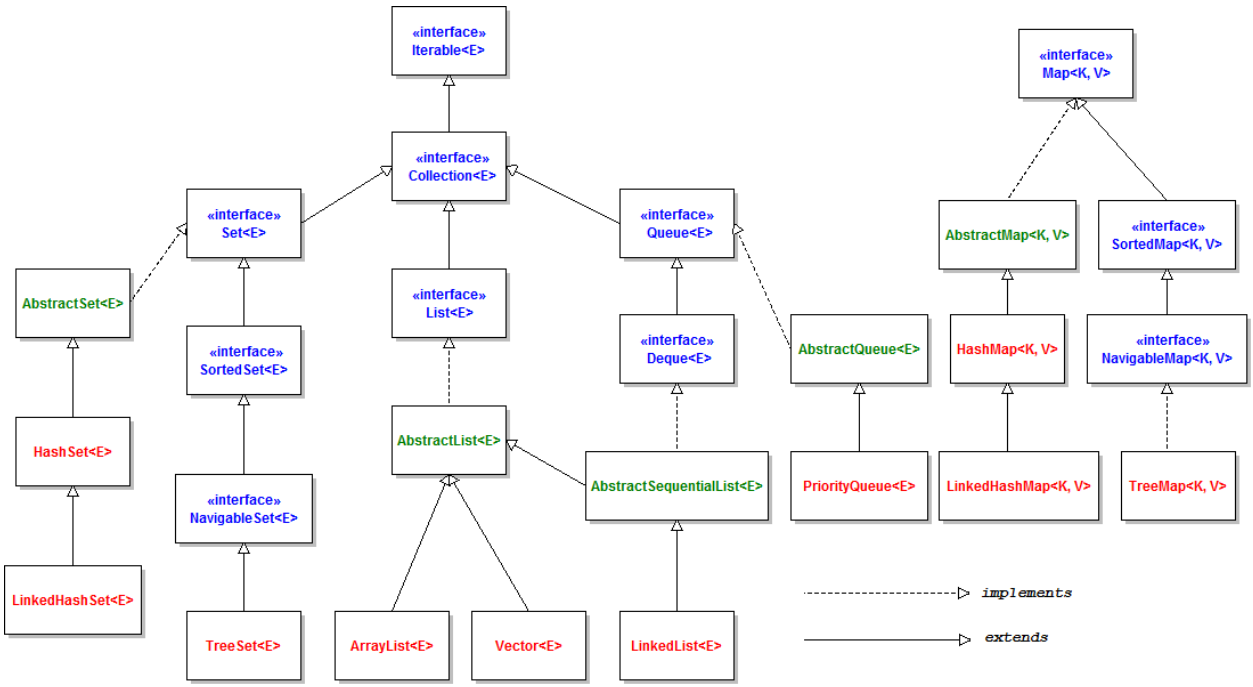
Use case	Unique data	Ordered unique	Sorted unique

🔹 Map in Java Collection Framework — HashMap, LinkedHashMap & TreeMap (Ultra-Detailed & Interview-Ready)

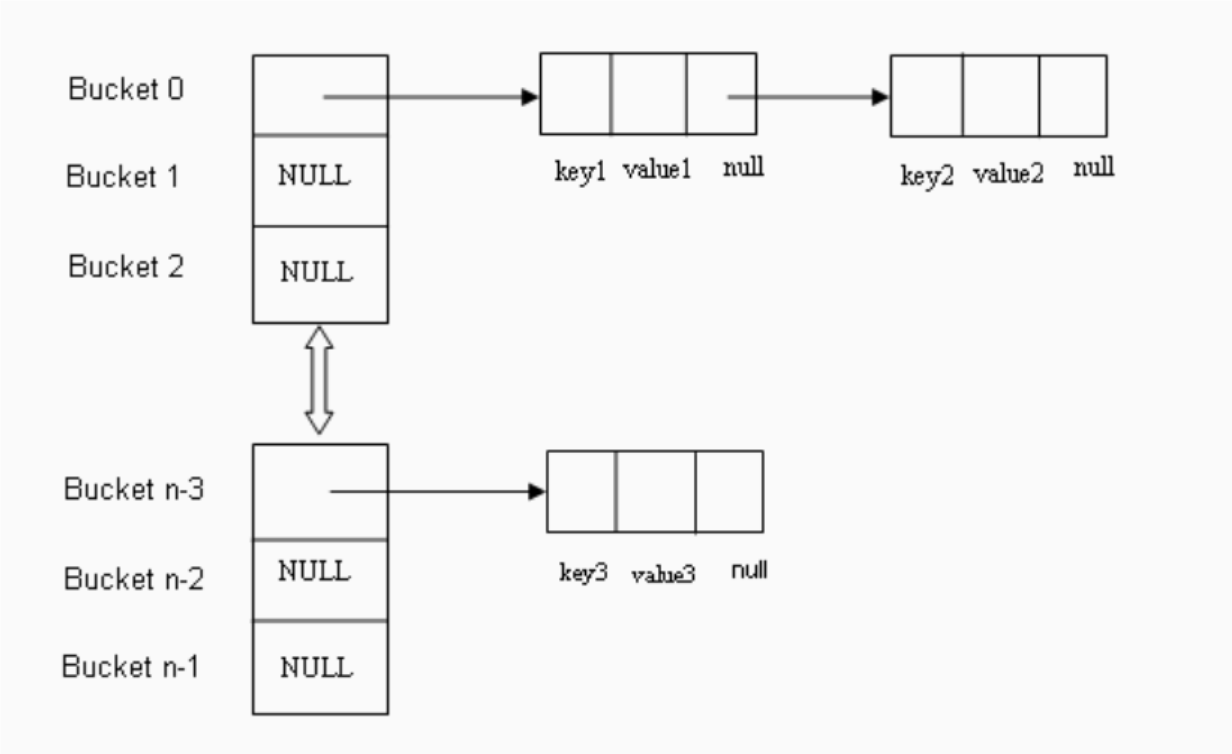
In Java, **Map** is **NOT a Collection**.
It stores data in **key–value pairs**.

👉 **Key must be unique**
👉 **Value can be duplicate**

```
key → value
```



Property	HashMap	LinkedHashMap	TreeMap
Time Complexity(Big O notation) Get, Put, ContainsKey and Remove method	O(1)	O(1)	O(1)
Iteration Order	Random	Sorted according to either Insertion Order of Access Order (as specified during construction)	Sorted according to either natural Order of keys or comparator(as specified during construction)
Null Keys	allowed	allowed	Not allowed if keys uses Natural Ordering or Comparator does not support comparison on null Keys.
Interface	Map	Map	Map, SortedMap and NavigableMap
Synchronization	None, use COllections.synchronizedMap()	None, use COllections.synchronizedMap()	None, use COllections.synchronizedMap()
Data Structure	List of buckets, if more than 8 entries in bucket then Java 8 will switch to balanced tree from linked list	Doubly Linked List of Buckets	Red-Black(a kind of self-balancing binary searc tree) implementation of Binary Tree. This data structure offers O(log n) for insert, Delte and Search operations and O(n) space complexity.
Applications	General Purpose, fast retrieval, non-synchronized. ConcurrentHashMap can be used where concurrency is involved.	Can be used for LRU cache, other places where insertion or access order matters	Algorithms where Sorted or Navigable features are required. For example, find among the list of amployees whose salary is next to given employee, Range Search, etc.
Requirements for Keys	Equals() and hashCode() needs to be overwritten.	Equals() and hashCode() needs to be overwritten.	Comparotor needs to be supplied for key implementation, otherwise natural order will be used to sort the keys.



🔹 Map Hierarchy (IMPORTANT)

```
Object
 └── Map (Interface)
      ├── HashMap
      ├── LinkedHashMap
      ├── TreeMap
      └── Hashtable (Legacy)
```

🔹 1. HashMap (Most Important & Most Used)

✅ What is HashMap?

HashMap stores data using **hashing** for very fast access.

✔ No order
✔ Fastest
✔ One null key, multiple null values

🔹 Creating HashMap (Best Practice)

```
Map<Integer, String> map = new HashMap<>();
```

🔹 Basic Operations

🔸 put() → Insert / Update

```
map.put(1, "Java");
map.put(2, "Python");
map.put(1, "C++"); // overwrites value
```

📌 **Keys are unique**
📌 If key already exists → value is replaced

🔸 get() → Fetch value

```
System.out.println(map.get(1)); // C++
System.out.println(map.get(5)); // null
```

🔹 ⭐ getOrDefault() (VERY IMPORTANT)

❓ Problem with get()

```
map.get(5); // returns null (may cause NullPointerException)
```

✅ Solution: getOrDefault()

```
String value = map.getOrDefault(5, "Not Found");
System.out.println(value); // Not Found
```

✔ If key exists → returns value
✔ If key does NOT exist → returns default value

🔹 Real Use Case (Frequency Count 🔥)

```
int[] arr = {1, 2, 2, 3, 1, 2};
Map<Integer, Integer> freq = new HashMap<>();

for (int x : arr) {
    freq.put(x, freq.getOrDefault(x, 0) + 1);
}
System.out.println(freq);
```

📌 **MOST ASKED INTERVIEW PATTERN**

🔹 containsKey() vs containsValue()

```
map.containsKey(1);     // true
map.containsValue("Java"); // false
```

✔ `containsKey()` is fast
❌ `containsValue()` is slow (O(n))

🔹 Removing Elements

```
map.remove(2);
```

🔹 Accessing Keys & Values (VERY IMPORTANT ⭐)

🔸 1. Access Only Keys

```
for (Integer key : map.keySet()) {
    System.out.println(key);
}
```

🔸 2. Access Only Values

```
for (String value : map.values()) {
    System.out.println(value);
}
```

🔸 3. Access Key + Value Together (BEST ⭐)

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
}
```

📌 **Most efficient & recommended**

🔸 4. Java 8 forEach()

```
map.forEach((k, v) -> System.out.println(k + " = " + v));
```

🔹 Internal Working of HashMap (🔥 MOST IMPORTANT)

▸ Step-by-step put(key, value)

1. `hashCode()` of key is called
2. Hash is converted into **bucket index**
3. If bucket empty → insert

4. If collision:
    - `equals()` is called
    - Same key → value replaced
    - Different key → added to bucket

▸ Java 8 Optimization

- Bucket structure:
    - **LinkedList** (default)
    - Converts to **Red-Black Tree** if:
        - Bucket size > **8**
        - Improves worst-case time

🔹 Time Complexity

Operation	Average	Worst
put	O(1)	O(log n)
get	O(1)	O(log n)
remove	O(1)	O(log n)

🔹 2. LinkedHashMap (Order Preserved)

✅ What is LinkedHashMap?

LinkedHashMap is a HashMap that **maintains insertion order**.

✔ Predictable iteration order
✔ Slightly slower than HashMap

🔹 Example

```
Map<Integer, String> map = new LinkedHashMap<>();

map.put(3, "C");
map.put(1, "A");
map.put(2, "B");

System.out.println(map); // {3=C, 1=A, 2=B}
```

🔹 Internal Working

- HashMap + **Doubly Linked List**
- DLL maintains order

🔹 Use Case

✔ Cache
✔ Recently viewed items
✔ Ordered output required

🔹 3. TreeMap (Sorted Map)

✅ What is TreeMap?

TreeMap stores data in **sorted order of keys**.

✔ Sorted automatically
✔ Uses **Red-Black Tree**
❌ Slower than HashMap

🔹 Example (Natural Sorting)

```
Map<Integer, String> map = new TreeMap<>();

map.put(30, "C");
map.put(10, "A");
map.put(20, "B");

System.out.println(map); // {10=A, 20=B, 30=C}
```

🔹 TreeMap with Custom Sorting

```
Map<Integer, String> map =
    new TreeMap<>((a, b) -> b - a);

map.put(10, "A");
map.put(30, "C");
map.put(20, "B");

System.out.println(map); // descending order
```

🔹 TreeMap Special Methods (INTERVIEW ⭐)

```
map.firstKey();
map.lastKey();
map.higherKey(20);
map.lowerKey(20);
map.subMap(10, 30);
```

📌 Used in **range queries**

🔹 TreeMap Properties

Feature	TreeMap
Order	Sorted
Null key	❌ Not allowed
Performance	O(log n)
Internal DS	Red-Black Tree

🔥 HashMap vs LinkedHashMap vs TreeMap (FINAL COMPARISON)

Feature	HashMap	LinkedHashMap	TreeMap
Order	❌ No	✅ Insertion	✅ Sorted
Speed	✅ Fastest	⚠ Medium	❌ Slow
Null key	✅ One	✅ One	❌ No
Internal DS	Hashing	Hashing + DLL	Red-Black Tree
Use case	Fast lookup	Ordered data	Sorted data

🔹 Custom Object as Key (VERY IMPORTANT ⭐)

For HashMap / LinkedHashMap

Must override:

```
hashCode()
equals()
```

For TreeMap

Must implement:

`Comparable`

or provide:

`Comparator`

🔹 Real-Life Use Cases

✔ UserID → User
✔ Word → Frequency (`getOrDefault`)
✔ Cache → LinkedHashMap
✔ Leaderboard → TreeMap

🔹 One-Line Summary (Exam Ready ✅)

> **HashMap provides fastest key-value storage without order, LinkedHashMap maintains insertion order, and TreeMap maintains sorted order using a Red-Black Tree, with methods like getOrDefault(), entrySet(), and range queries playing a crucial role in real-world usage.**

🔹 `Collections` Utility Class (MOST USED)

Used when working with **List / Collection**

```
import java.util.Collections;
```

⭐ Most Daily-Used Methods

1️⃣ `Collections.sort()`

```
Collections.sort(list);
```

✔ Sorts list in ascending order

```
Collections.sort(list, Collections.reverseOrder());
```

✔ Sort descending

2️⃣ Collections.reverse()

```
Collections.reverse(list);
```

✔ Reverses order

3️⃣ Collections.max() / Collections.min()

```
int max = Collections.max(list);
int min = Collections.min(list);
```

4️⃣ Collections.frequency()

```
int count = Collections.frequency(list, 10);
```

✔ Count occurrences (VERY useful)

5️⃣ Collections.shuffle()

```
Collections.shuffle(list);
```

✔ Randomize order (games, quizzes)

6️⃣ Collections.fill()

```
Collections.fill(list, 0);
```

✔ Replace all elements with same value

7️⃣ `Collections.copy()`

```
Collections.copy(destList, srcList);
```

⚠ Destination must be same size

8️⃣ `Collections.binarySearch()`

```
int index = Collections.binarySearch(list, 20);
```

⚠ List **must be sorted**

9️⃣ `Collections.unmodifiableList()`

```
List<Integer> safeList = Collections.unmodifiableList(list);
```

✔ Read-only list (no modification)

🔹 `Arrays` Utility Class (FOR ARRAYS)

```
import java.util.Arrays;
```

⭐ Most Daily-Used Methods

1️⃣ `Arrays.sort()`

```
Arrays.sort(arr);

Arrays.sort(arr, Collections.reverseOrder()); // Integer[]
```

2️⃣ Arrays.toString()

```
System.out.println(Arrays.toString(arr));
```

3️⃣ Arrays.asList()

```
List<Integer> list = Arrays.asList(1,2,3);
```

⚠ Fixed-size list (no add/remove)

4️⃣ Arrays.equals()

```
Arrays.equals(arr1, arr2);
```

5️⃣ Arrays.fill()

```
Arrays.fill(arr, 0);
```

6️⃣ Arrays.copyOf()

```
int[] newArr = Arrays.copyOf(arr, arr.length);
```

7️⃣ Arrays.copyOfRange()

```
int[] part = Arrays.copyOfRange(arr, 1, 4);
```

8️⃣ Arrays.binarySearch()

```
int idx = Arrays.binarySearch(arr, 20);
```

⚠ Array must be sorted

🔹 `Comparator` Helper Methods (VERY COMMON)

```
import java.util.Comparator;
```

1️⃣ Sort by value

```
Collections.sort(list, Comparator.naturalOrder());
Collections.sort(list, Comparator.reverseOrder());
```

2️⃣ Sorting Map by Key

```
Map<Integer,String> map = new TreeMap<>(map);
```

3️⃣ Sorting Map by Value

```
map.entrySet()
   .stream()
   .sorted(Map.Entry.comparingByValue())
   .forEach(System.out::println);
```

🔹 `Stream` Short Utility Operations (BONUS)

Convert Collection to List

```
list.stream().toList();
```

Filter

```
list.stream().filter(x -> x > 10).toList();
```

🔥 MOST USED IN REAL LIFE (MEMORIZE ONLY THIS)

✅ `Collections.sort()`
✅ `Collections.reverse()`

✅ Collections.max()
✅ Collections.frequency()
✅ Arrays.sort()
✅ Arrays.toString()
✅ Arrays.asList()
✅ Arrays.fill()